

# SANJEEVANI

## 3-Member Engineering Workflows & AI-Agent Handoff Specification

**Purpose:** Give each team member—and any coding AI/agent—a precise, non-overlapping understanding of ownership, interfaces, implementation order, acceptance criteria, Git workflow, and integration responsibilities.

**Project:** SANJEEVANI — The Self-Healing Pharma Supply Chain Network

**Core principle:** Sense → Verify → Understand → Scenario → Simulate → Optimize → Approve → Execute → Audit → Learn.

**Architecture rule:** Six disciplined MVP agents implement the required capabilities; do not turn the conceptual 16-role roster into 16 independent agents.

### 1. NON-NEGOTIABLE TEAM ARCHITECTURE

The team works in **one Git repository**. Each member owns a vertical slice, but all three integrate through explicit JSON/Pydantic-style contracts. The critical path is one polished end-to-end demo, not three independent subsystems.

| Member   | Primary ownership                                       | Question answered                   | MVP agents |
|----------|---------------------------------------------------------|-------------------------------------|------------|
| Member 1 | Sense + Verify + Scenario + Network Impact              | What happened and what is affected? | A1, A2, A3 |
| Member 2 | Digital Twin + Recovery + Optimization + Routing        | What should we do?                  | A4         |
| Member 3 | Compliance + HITL + SAP execution + Audit/Learning + UI | Can we safely execute it?           | A5, A6     |

**Important:** A2 combines verification + scenario; A3 combines graph + impact; A4 combines inventory/logistics/supplier/optimization; A5 combines compliance + approval; A6 combines audit + learning. This follows the dossier's intended MVP compression.

## 2. SHARED REPOSITORY & OWNERSHIP

```
sanjeevani/  
■■■ README.md  
■■■ .env.example  
■■■ requirements.txt  
■■■ package.json  
■■■ config/  
■ ■■■ settings.yaml  
■ ■■■ policies.yaml  
■ ■■■ scenarios.yaml  
■■■ data/  
■ ■■■ network/  
■ ■■■ products/  
■ ■■■ suppliers/  
■ ■■■ shipments/  
■ ■■■ events/  
■ ■■■ telemetry/  
■■■ backend/  
■ ■■■ agents/  
■ ■■■ simulation/  
■ ■■■ optimization/  
■ ■■■ governance/  
■ ■■■ sap/  
■ ■■■ audit/  
■ ■■■ api/  
■■■ frontend/  
■■■ tests/  
■■■ scripts/  
■■■ docs/  
■■■ demo/
```

| Area                                              | M1     | M2     | M3     |
|---------------------------------------------------|--------|--------|--------|
| data/                                             | OWNER  |        |        |
| agents/sensing + verification + scenario + impact | OWNER  |        |        |
| graph/                                            | OWNER  |        |        |
| simulation/                                       |        | OWNER  |        |
| optimization/ + routing/                          |        | OWNER  |        |
| governance/                                       |        |        | OWNER  |
| sap/                                              |        |        | OWNER  |
| audit/ + learning/                                |        |        | OWNER  |
| frontend/                                         |        |        | OWNER  |
| api/                                              |        |        | OWNER  |
| config/contracts/tests/docs                       | SHARED | SHARED | SHARED |

## 3. SHARED DATA CONTRACTS — FREEZE THESE FIRST

No member should invent a private schema that another member must reverse-engineer. These are the minimum integration objects. Use stable field names and version them if a breaking change is unavoidable.

### Event

```
{"event_id":"EVT-001","type":"COLD_CHAIN_EXCURSION","severity":"HIGH","confidence":0.96,"timestamp":"ISO-8601","source":"IoT"}
```

### Impact

```
{"event_id":"EVT-001","affected_nodes":[],"affected_shipments":[],"revenue_at_risk":0,"stockout_probability":0,"service_level_risk":0}
```

### RecoveryPlan

```
{"plan_id":"PLAN-001","strategy":"REROUTE_MUMBAI_AIR","cost":0,"service_level":0,"resilience_score":0,"temperature_risk":0,"recovery_time_hours":0,"compliance_status":"PENDING"}
```

## Approval

```
{"plan_id":"PLAN-001","approved":true,"approved_by":"Meera","timestamp":"ISO-8601"}
```

## ExecutionReceipt

```
{"plan_id":"PLAN-001","ibp":"SUCCESS","tm":"SUCCESS","ariba":"SUCCESS","timestamp":"ISO-8601"}
```

## Outcome

```
{"plan_id":"PLAN-001","predicted":{},"actual":{},"delta":{},"learning_note":""}
```

## 4. MEMBER 1 — AI / DATA / GRAPH LEAD

**Mission:** Build the upstream intelligence layer. Convert raw signals into a verified disruption, scenario, network impact, and machine-readable handoff to Member 2.

**Own:** A1 Sensing; A2 Verification & Scenario; A3 Network Impact; synthetic pharma network; event data; graph/network reasoning; confidence/provenance; impact report.

**Do NOT own:** recovery optimization, SAP execution, approval UI.

### Workflow

Signals → normalize → classify → verify/deduplicate → confidence/staleness → create scenario → traverse network → calculate affected shipments/products → calculate stockout/service/revenue risk → emit **Impact** contract.

### Implementation order

1. Create deterministic synthetic 12-node pharma network: suppliers, plants, ports, warehouses, lanes and customers.
2. Create scripted disruption feeds: Red Sea closure and biologics temperature excursion. Keep all demo inputs deterministic/offline.
3. Implement Sensing Agent: ingestion, normalization, event typing and source metadata.
4. Implement Verification: confidence, deduplication, freshness/staleness and severity.
5. Implement Scenario Agent: convert verified event into parameterized 3/10/30-day scenario objects.
6. Implement Network Impact: graph traversal and affected-node/shipments calculation.
7. Add impact metrics: revenue-at-risk, stockout probability, service-level risk.
8. Expose one clean function/API that returns the shared Impact object.
9. Write unit tests and one integration fixture consumed by Member 2.

### Acceptance criteria

- ✓ Given the seeded scenario, the system detects the intended event deterministically.
- ✓ Verification returns confidence, provenance and freshness fields.
- ✓ Network impact is reproducible from the same input.
- ✓ Output validates against the shared Impact schema.
- ✓ Member 2 can run your module without importing private internals.
- ✓ Every important number shown in the UI can be traced to source data or a deterministic calculation.

### AI-agent instruction

If you are an AI coding agent working for Member 1: inspect the existing repository before creating files; preserve existing working code; implement only Member 1 ownership; do not redesign the architecture; do not add agents; do not add real SAP integrations; create tests for every public contract; document every interface you expose; finish with exact run/test commands and a list of files changed.

## 5. MEMBER 2 — DIGITAL TWIN / OPTIMIZATION LEAD

**Mission:** Take Member 1's quantified impact and determine the best recovery plan using deterministic simulation and mathematical optimization.

**Own:** A4 Recovery; SimPy digital twin; three recovery strategies; Pyomo + HiGHS MILP; OR-Tools routing; cost/service/resilience/temperature-risk calculations.

**Do NOT own:** event sensing, final compliance gate, SAP API implementation, frontend.

### Workflow

Impact → construct disrupted network state → replay baseline → simulate WAIT / REROUTE VIA MUMBAI + AIR / EMERGENCY SOURCE → calculate KPIs → optimize → select best feasible plan → emit **RecoveryPlan**.

### Implementation order

1. Consume Member 1's Impact contract using a fixture first; do not wait for live integration.
2. Implement a deterministic SimPy digital twin for the 12-node network.
3. Implement baseline 'WAIT' strategy.
4. Implement 'REROUTE VIA MUMBAI + AIR'.
5. Implement 'EMERGENCY SOURCE'.
6. Define objective terms: recovery cost, service level, resilience score and temperature risk.
7. Implement Pyomo/HiGHS optimization with explicit constraints and bounds. The LLM must not perform the math.
8. Add OR-Tools routing only where it materially supports the demo route decision.
9. Return a ranked strategy table plus one RecoveryPlan object.
10. Add deterministic tests for feasibility, objective calculation and repeatability.

### Acceptance criteria

- ✓ All three strategies run from the same scenario input.
- ✓ Repeated runs with the same seed produce the same results.
- ✓ The optimizer never violates hard constraints.
- ✓ The recommended plan validates against the shared RecoveryPlan schema.
- ✓ The strategy table contains enough data for the UI to render cost/service/risk/resilience.
- ✓ No LLM-generated number is treated as authoritative when a deterministic engine can calculate it.

### AI-agent instruction

If you are an AI coding agent working for Member 2: first inspect Member 1's contract and current repository. Build the simulation/optimization layer against the contract fixture. Keep solver logic deterministic and testable. Never silently change upstream schemas. Do not add GNNs, real SAP APIs, model training, or extra agents. Report solver assumptions, constraints, objective terms, test commands, and integration points when finished.

## 6. MEMBER 3 — SAP / GOVERNANCE / UI LEAD

**Mission:** Turn the optimized recovery plan into a governed business action: validate it, ask for human approval, execute through SAP-shaped mocks, and produce an auditable result.

**Own:** A5 Compliance + HITL; CAP mocks for IBP/TM/Ariba; execution receipts; A6 Audit + Learning; React/Vite dashboard; approval screen; demo orchestration API.

**Do NOT own:** core optimization mathematics or upstream event detection.

### Workflow

RecoveryPlan → policy validation → compliance checks → risk tier → approval card → human approval → SAP-shaped execution → receipts → audit ledger → outcome/learning panel.

### Implementation order

1. Create backend API/orchestrator that can consume a RecoveryPlan fixture.
2. Implement policy engine: budget caps, parameter bounds, sanctions/trade/GDP-style checks and approval rules.
3. Implement approval state machine; consequential actions must pause for human approval.
4. Implement CAP Node.js mocks shaped like IBP scenario, TM freight and Ariba risk/procurement contracts.
5. Implement execution client with transactional sequencing and visible receipts.
6. Implement append-only decision ledger: event, reasoning metadata, scenario, recommendation, approval, action and outcome.
7. Implement learning/outcome view comparing predicted vs actual and showing a recalibration notice; do not pretend to have autonomous model retraining in MVP.
8. Build dashboard: loop HUD, network map, impact KPIs, strategy table, compliance panel, approval card, execution receipts, audit/learning.
9. Add offline demo mode and one-command reset.
10. Integrate all three members and freeze the demo path.

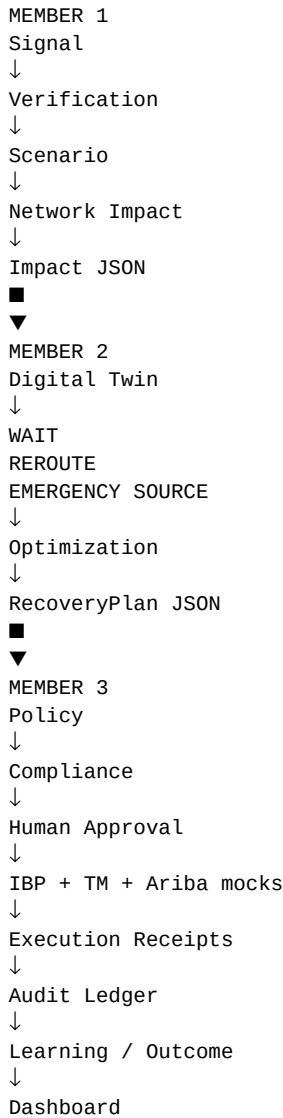
### Acceptance criteria

- ✓ Every execution is blocked until the required approval is present.
- ✓ Compliance failure prevents execution and explains why.
- ✓ IBP/TM/Ariba mocks return deterministic receipts.
- ✓ Audit record exists for every major loop stage.
- ✓ Frontend can render the complete journey from one seeded scenario.
- ✓ Demo runs without venue Wi-Fi or live APIs.

### AI-agent instruction

If you are an AI coding agent working for Member 3: inspect the repository and shared RecoveryPlan contract first. Preserve the governance boundary. Build API-faithful mocks rather than real SAP tenant integrations. Never allow a UI click to bypass policy validation. Keep the demo deterministic/offline. Do not add extra agents or features. Return exact start/reset/demo commands and identify every mock boundary clearly.

## 7. INTEGRATION FLOW — ALL THREE MEMBERS



### Integration rule

Each member must be able to test their portion with a frozen fixture from the previous member. Do not block development waiting for the previous module. Once the real module is ready, swap the fixture for the live call and run contract tests.

### End-to-end demo scenario

**Hero scenario:** Red Sea disruption + biologics cold-chain excursion. The twin compares WAIT, REROUTE VIA MUMBAI + AIR, and EMERGENCY SOURCE. The optimizer selects a feasible recovery plan. Compliance validates it. A human approves. The system posts to SAP-shaped IBP/TM/Ariba mocks. Receipts and the audit ledger close the loop.

## 8. GIT / COLLABORATION RULES

- ✓ **main** is always runnable.
- ✓ Each member works on a dedicated branch: member1-ai, member2-optimization, member3-sap-ui.
- ✓ Pull/rebase or merge the latest main before integration work.
- ✓ Small commits; one logical change per commit.
- ✓ Never commit secrets, API keys, .env files, generated databases or huge model files.

- ✓ Every PR must include: what changed, files changed, tests run, integration impact, and rollback note.
- ✓ Breaking contract changes require all three members to agree before merge.
- ✓ After Sep 27, no architecture changes unless required for a critical bug.

### **Suggested commit format**

```
feat(sensing): add disruption ingestion
feat(impact): add network risk calculation
feat(twin): add deterministic replay
feat(optimizer): add recovery MILP
feat(policy): add compliance gate
feat(sap): add TM mock execution
feat(ui): add approval console
test(integration): verify recovery contract
```

## 9. BUILD SCHEDULE

| Date      | Member 1                 | Member 2                | Member 3                      | Team gate               |
|-----------|--------------------------|-------------------------|-------------------------------|-------------------------|
| Sep 15–16 | Data/network + contracts | Simulation skeleton     | Repo/API/UI skeleton          | Architecture frozen     |
| Sep 17    | Sensing + verification   | Twin baseline           | Dashboard base                | Event → verified        |
| Sep 18    | Scenario generation      | 3 strategies            | Policy skeleton               | Scenario contract works |
| Sep 19    | Graph + impact           | Simulation metrics      | Approval UI                   | Impact → strategy       |
| Sep 20    | Integration tests        | Optimizer               | SAP mock skeleton             | First E2E               |
| Sep 21–22 | Round 1 support/Q&A;     | Round 1 support/Q&A;    | Round 1 support/Q&A;          | Submission + rehearsal  |
| Sep 23    | Seed final data          | Environment check       | CAP/HANA/API scaffolding      | All machines run        |
| Sep 24    | A1+A2                    | Twin                    | UI/SAP mocks                  | Sense → Scenario        |
| Sep 25    | A3                       | SimPy replay            | Impact visualization          | Impact → Twin           |
| Sep 26    | Support tests            | MILP + routing          | Policy v1                     | RecoveryPlan            |
| Sep 27    | Fix/integration          | Fix/integration         | Approval + execution + ledger | FULL LOOP WORKS         |
| Sep 28    | Polish intelligence      | Polish strategy visuals | Polish dashboard              | Demo-ready              |
| Sep 29    | Freeze                   | Freeze                  | Freeze                        | 5 rehearsals + backup   |
| Sep 30    | Presentation             | Presentation            | Presentation                  | Finale                  |

The source dossier's MVP sprint places environment/data on Sep 23, LangGraph A1/A2 on Sep 24, A3 + twin on Sep 25, optimization + policy on Sep 26, end-to-end approval/execution/audit on Sep 27, polish on Sep 28, and rehearsals/fallback on Sep 29. This team plan assigns those activities to the three owners above.

## 10. DEFINITION OF DONE

- ✓ **Functional:** one complete deterministic scenario runs from signal to audit.
- ✓ **Governed:** no consequential SAP-shaped action bypasses policy + human approval.
- ✓ **Explainable:** every recommendation has visible reasons and numerical evidence.
- ✓ **Reproducible:** seeded data and offline mode produce the same demo result.
- ✓ **Integrated:** all shared contracts validate automatically.
- ✓ **Demoable:** presenter can reset the system and rerun the scenario quickly.
- ✓ **Defensible:** team can explain what is real, what is simulated, and what is mocked.
- ✓ **Scope-safe:** no real tenant integration, no trained forecasting model, no GNN cascade, no extra agents in MVP.

## 11. AI AGENT HANDOFF TEMPLATE

Copy the following section into a coding agent before asking it to work on any member's area.

PROJECT: SANJEEVANI  
 REPOSITORY: One shared Git repository  
 YOUR ROLE: [MEMBER 1 / MEMBER 2 / MEMBER 3]

FIRST:

1. Inspect the entire repository and current git status.
2. Read README, docs/architecture.md, shared contracts and relevant tests.
3. Identify existing implementation before creating anything.
4. Do not restart or replace working components.

YOU OWN:

[Paste the member-specific ownership section from this PDF.]

YOU DO NOT OWN:

[Paste the exclusions.]

ARCHITECTURE:

Sense → Verify → Scenario → Impact → Simulate → Optimize → Policy → Approve → Execute → Audit → Learn.

RULES:

- Preserve shared contracts.
- Do not add agents without team approval.
- Do not silently change schemas.
- Prefer deterministic/offline demo behavior.
- Use mocks where the architecture specifies mocks.
- Keep LLMs for reasoning/orchestration; deterministic engines perform calculations.
- Write tests for public interfaces.
- Do not introduce unrelated refactors.
- Never commit secrets.

TASK:

[ONE specific task]

DONE WHEN:

[Acceptance criteria]

BEFORE FINISHING:

- Run tests.
- Run the relevant demo path.
- Check git diff.
- List files changed.
- List commands used to test.
- Explain any assumptions.
- Explain any integration dependency for the next member.
- Give the exact next command a teammate/agent should run.

## 12. DEMO RUNBOOK

| Time       | Screen/action                            | Owner |
|------------|------------------------------------------|-------|
| 0:00–1:30  | Persona + disruption + signal arrives    | M1    |
| 1:30–3:00  | Verification + scenario + network impact | M1    |
| 3:00–5:30  | Digital twin replays 3 strategies        | M2    |
| 5:30–7:00  | Optimizer selects recovery plan          | M2    |
| 7:00–8:00  | Compliance/policy panel                  | M3    |
| 8:00–9:00  | Human approval → SAP mock receipts       | M3    |
| 9:00–9:40  | Outcome + learning                       | M3    |
| 9:40–10:00 | Close the Sense-Decide-Act loop          | All   |

Critical WOW moment: one human approval triggers multiple governed SAP-shaped actions, followed by visible receipts and an audit record. The dossier specifically recommends rehearsing this approval click until the loop feels closed.

## 13. SCOPE CONTROL — RED FLAGS

| If an agent suggests...                              | Decision                                  |
|------------------------------------------------------|-------------------------------------------|
| 16 separate agents                                   | REJECT — keep six MVP agents              |
| Real SAP tenant integration                          | DEFER — use API-faithful CAP mocks        |
| Training a forecasting model                         | DEFER — zero-shot/scripted inputs for MVP |
| GNN cascade                                          | DEFER to research layer                   |
| More disruption scenarios before hero scenario works | REJECT                                    |
| Online-only external APIs                            | REJECT — offline seeded mode required     |
| LLM calculating optimization numbers                 | REJECT — solver/math engine decides       |
| UI bypassing policy/approval                         | REJECT                                    |

## 14. FINAL TEAM CHECKLIST

- One GitHub repo created and cloned by all 3 members.
- Branches and ownership agreed.
- Shared contracts frozen.
- 12-node network + hero scenario seeded.
- Member 1 can produce Event/Impact.
- Member 2 can produce RecoveryPlan.
- Member 3 can approve/execute/audit.
- Full loop runs offline.
- Dashboard shows the loop visually.
- Reset command works.
- Tests pass.
- Five full rehearsals completed.
- Backup video/static walkthrough ready.
- Each member can explain their own subsystem and the two interfaces around it.

## SOURCE BASIS

This specification is derived from the uploaded SANJEEVANI SAP Hackfest 2026 Unified Dossier, especially its three-layer MVP design, six-agent MVP compression, component/SAP mapping, governance requirements, demo sequence, and Sep 23–29 implementation schedule. Where this document assigns work between three people, that is a team-structure recommendation built from the dossier's architecture; it is not presented as a quoted source assignment.